Day 2 — Linear Regression

Objective

Understand linear regression from the equation to optimization and practical implementation.

Topics and detailed notes

- Hypothesis: prediction is a weighted combination of input features plus an intercept.
- Coefficient interpretation depends on feature scale and other variables in the model.
- Residual = actual value − predicted value.
- Ordinary least squares minimizes squared residuals.
- Normal equation gives a closed-form solution under suitable conditions.
- Gradient descent provides an iterative optimization route.
- Multicollinearity can make coefficients unstable.
- Outliers can strongly affect squared-error objectives.
- Linear regression can still be useful when the real world is not perfectly linear because it can be an interpretable baseline.

Mathematics / formulas to know

1. Linear regression model

For one feature:

$$\hat{y} = wx + b$$

For multiple features:

$$\hat{y} = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

Matrix form:

$$\hat{y} = Xw + b$$

2. Residual / prediction error

$$e_i = y_i - \hat{y}_i$$

where:

- y_i = actual value
- $\hat{y}_i$ = predicted value

3. Mean Squared Error

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

This is one of the most important formulas in regression.

4. Root Mean Squared Error

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

RMSE has the **same units as the target variable.**

5. Mean Absolute Error

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

6. Cost function

For linear regression, a common cost function is:

$$J(w,b) = \frac{1}{2n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

The factor $1/2$ makes the derivative cleaner.

7. Gradient of the cost function

For w:

$$\frac{\partial J}{\partial w} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)x_i$$

For b:

$$\frac{\partial J}{\partial b} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)$$

8. Gradient Descent

$$w := w - \alpha\frac{\partial J}{\partial w}$$

$$b := b - \alpha\frac{\partial J}{\partial b}$$

where α is the learning rate.

9. R^2

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

Interpretation:

- $R^2 = 1$ → perfect predictions
- $R^2 = 0$ → no improvement over predicting the mean
- $R^2 < 0$ → worse than the mean baseline

10. Normal Equation

For linear regression:

$$\theta = (X^T X)^{-1} X^T y$$

You should understand what every matrix represents, because this connects your linear algebra knowledge directly to ML.

Implementation pattern

```
Load data → define X/y → split → establish baseline → train → predict → calculate metrics
→ inspect residuals → document.
```

Experiments to perform

- Implement the normal equation with NumPy.
- Implement gradient descent for a small regression problem.
- Train the sklearn equivalent and compare parameters/predictions.
- Keep preprocessing inside the correct training workflow.

End-of-day deliverables

- Vary learning rate.
- Vary feature scaling.
- Compare normal equation vs gradient descent.
- Add an outlier and observe coefficient/error changes.

Mastery questions

1. House-price or salary regression notebook.
2. From-scratch implementation.
3. sklearn comparison.
4. Residual plot and metric table.

PLAN

Welcome to Day 2. Today, we elevate our engineering from raw optimization loops into a formal, structured predictive model: **Linear Regression**.

As a Senior ML Engineer, I look at Linear Regression not just as a statistical tool, but as the ultimate baseline algorithm. When you expand your Retro Reload digital store database, you might eventually want to predict continuous variables—like forecasting next month's game sales based on marketing spend and historical downloads. Linear Regression is exactly how you establish the mathematical baseline for that problem before you ever touch a complex neural network.

Here is our master plan for Day 2, completely aligned with your Global ML Workflow and strictly incorporating every concept from your provided syllabus.

Phase 1: The Theory & Architecture of Linear Regression

Before we code, we define the mathematical reality. This phase maps to **Step 1 (Define the problem)** and **Step 2 (Define target and prediction unit)**.

- **The Hypothesis:** Our prediction is simply a weighted combination of input features plus an intercept.
- **The Residual (Prediction Error):** The absolute truth of how wrong we are. It is calculated as the actual value minus the predicted value ($e_{i}=y_{i}-\hat{y}_{i}$).
- **Ordinary Least Squares (OLS):** The mathematical objective that minimizes those squared residuals across the entire dataset.
- **Two Paths to Optimization:**
    - **Gradient Descent:** The iterative optimization route we built yesterday. We take small steps down the error curve.
    - **The Normal Equation:** A closed-form algebraic solution that calculates the exact perfect weights in one massive matrix operation.
- **System Vulnerabilities:**
    - *Multicollinearity:* If two features are highly correlated (e.g., tracking both "hours played" and "minutes played"), it can make the mathematical coefficients unstable.
    - *Outliers:* Because our objective function squares the errors, a single massive outlier can strongly affect and skew the model's parameters.

Phase 2: The Mathematical Language (Formulas to Know)

This is the engine room for **Step 8 (Build preprocessing)** and **Step 9 (Train candidate models)**. You must know these formulas:

The Models:

- Single Feature: $\hat{y}=wx+b$
- Multiple Features: $\hat{y}=w_{1}x_{1}+w_{2}x_{2}+\cdot\cdot\cdot+w_{n}x_{n}+b$
- Matrix Form: $\hat{y}=Xw+b$

The Loss & Optimization (Gradient Descent):

- Cost Function (MSE variant): $J(w,b)=\frac{1}{2n}\sum_{i=1}^{n}(\hat{y}_{i}-y_{i})^{2}$ (The factor $1/2$ makes the calculus derivative cleaner).
- Gradient for w: $\frac{\partial J}{\partial w}=\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_{i}-y_{i})x_{i}$
- Gradient for b: $\frac{\partial J}{\partial b}=\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_{i}-y_{i})$
- Update Rules: $w:=w-\alpha\frac{\partial J}{\partial w}$ and $b:=b-\alpha\frac{\partial J}{\partial b}$

The Loss & Optimization (Normal Equation):

- Matrix Formula: $\theta=(X^{T}X)^{-1}X^{T}y$

The Evaluation Metrics:

- Mean Squared Error (MSE): $\frac{1}{n}\sum_{i=1}^{n}(y_{i}-\hat{y}_{i})^{2}$

- Root Mean Squared Error (RMSE): $\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_{i}-\hat{y}_{i})^{2}}$ (This returns the error in the exact same units as your target variable).
- Mean Absolute Error (MAE): $\frac{1}{n}\sum_{i=1}^{n}\vert{}y_{i}-\hat{y}_{i}\vert{}$

- R^{2} Score: $1-\frac{\sum(y_{i}-\hat{y}_{i})^{2}}{\sum(y_{i}-\overline{y})^{2}}$. This dictates variance explanation. An $R^{2}=1$ is perfect predictions, $R^{2}=0$ means the model is no better than predicting the mean, and $R^{2}<0$ means your model is actively worse than a dumb baseline.

Phase 3: The Day 2 Implementation Pattern & Tasks

We will write code that strictly follows this pattern: Load data $\rightarrow$ define X/y $\rightarrow$ split $\rightarrow$ establish baseline $\rightarrow$ train $\rightarrow$ predict $\rightarrow$ calculate metrics $\rightarrow$ inspect residuals $\rightarrow$ document.

Here are your distinct tasks to execute today.

Task 1: The Dual-Engine Implementation (From Scratch)

- You will write the Normal Equation ($\theta=(X^{T}X)^{-1}X^{T}y$) entirely in NumPy.
- You will implement Gradient Descent for a small regression problem.
- *Experiment:* Compare the weights generated by the Normal Equation vs. Gradient Descent. They should ideally be identical.

Task 2: The Production Pipeline (sklearn)

- You will train the scikit-learn equivalent model (LinearRegression) and compare its parameters and predictions against your manual implementations.
- You will enforce strict data isolation, ensuring all preprocessing (like scaling) stays inside the correct training workflow boundaries.

Task 3: Vulnerability & Metric Experiments

- *Experiment:* Vary the learning rate and feature scaling to observe system limits.
- *Experiment:* Inject a massive outlier into your dataset and observe how drastically the coefficients and squared-error changes.
- Calculate and format a metric table showing MSE, RMSE, MAE, and R^{2}.
- Generate a Residual Plot to visually inspect where the model is failing.

Phase 4: Final Deliverables

By the end of today, you will have compiled a comprehensive notebook (focused on House-price or salary regression) containing:

1. The from-scratch implementations (Gradient Descent & Normal Equation).
2. The sklearn production comparison.
3. The Residual plot and evaluation metric table.